A Comprehensive Survey of Reinforcement Learning Algorithms for Large Language Model Training: PPO, DPO, GRPO, and DAPO

Aitachi 44158892@qq.com

***Abstract*—Reinforcement learning (RL) has become a cornerstone technique for aligning and enhancing large language models (LLMs). This survey provides a comprehensive and systematic analysis of four prominent RL algorithms used in LLM training: Proximal Policy Optimization (PPO), Direct Preference Optimization (DPO), Group Relative Policy Optimization (GRPO), and Dynamic Advantage Policy Optimization (DAPO). We present rigorous mathematical formulations for each algorithm, compare their architectural designs and computational requirements, and evaluate their empirical performance on mathematical reasoning benchmarks including AIME 2024. Our analysis reveals a clear evolutionary trajectory from value-based methods (PPO) to group-based methods (GRPO/DAPO), with each generation addressing critical limitations of its predecessor. Specifically, DAPO achieves 50% accuracy on AIME 2024—a 67% relative improvement over naive GRPO—through three key innovations: asymmetric Clip-Higher, dynamic sampling, and token-level loss normalization. We provide practical algorithm selection guidelines and identify open challenges for future research. To our knowledge, this is the first survey that unifies these four algorithms under a common mathematical framework with side-by-side experimental evaluation.**

***Index Terms*—Reinforcement Learning, Large Language Models, PPO, DPO, GRPO, DAPO, RLHF, Alignment, Mathematical Reasoning**

I. Introduction

The emergence of large language models (LLMs) such as GPT-4 [12], LLaMA [11], and Qwen2.5 [23] has fundamentally transformed natural language processing, enabling unprecedented capabilities in mathematical reasoning, code generation, scientific analysis, and creative writing. However, models trained solely via next-token prediction on large corpora often produce outputs that misalign with human preferences regarding helpfulness, safety, and factual accuracy.

Reinforcement Learning from Human Feedback (RLHF) [6], [7] has emerged as the dominant paradigm for bridging this alignment gap. The canonical RLHF pipeline consists of three stages: (1) Supervised Fine-Tuning (SFT) on high-quality demonstrations; (2) Reward Model (RM) training on human preference comparisons; and (3) RL-based policy optimization using the trained reward model. While stages 1 and 2 are relatively standardized, the choice of RL algorithm in stage 3 has profound implications for training stability, computational efficiency, and final model performance.

The evolution of RL algorithms for LLM training has accelerated dramatically:

- **PPO** [1] (2017): The foundational algorithm, requiring both a policy network (Actor) and a value network (Critic). It has been the workhorse behind InstructGPT [6] and ChatGPT, but demands significant GPU memory for the dual-network architecture.
- **DPO** [3] (2023): A paradigm shift that eliminates explicit reward modeling by deriving a closed-form loss function from preference pairs. DPO simplifies the training pipeline but requires pre-collected preference data.
- **GRPO** [4], [21] (2025): Introduced by DeepSeek, GRPO eliminates the value network by using group-level reward statistics as the advantage baseline. This reduces GPU memory by approximately 50% compared to PPO.
- **DAPO** [5] (2025): Proposed by ByteDance, DAPO extends GRPO with three key innovations—asymmetric Clip-Higher, dynamic sampling, and token-level loss—achieving state-of-the-art 50% accuracy on AIME 2024.

This survey makes the following contributions:

1) A **unified mathematical framework** with consistent notation across all four algorithms, enabling direct cross-algorithm comparison of objective functions and design choices.
2) A **systematic architectural comparison** covering network requirements, clipping strategies, loss granularity, and KL constraint mechanisms.
3) An **experimental evaluation** on mathematical reasoning benchmarks with quantitative results for training efficiency, model quality, and resource utilization.
4) **Practical guidelines** for algorithm selection based on task characteristics and resource constraints.

The remainder of this paper is organized as follows: Section II establishes background and notation; Sections III–VI present each algorithm in detail; Section VII provides comparative analysis; Section VIII presents experimental results; Section IX discusses implications and future directions; and Section X concludes.

II. Background and Preliminaries

A. Reinforcement Learning Formulation for LLMs

In the context of LLM alignment, the training process is formulated as a contextual bandit problem rather than

a full Markov Decision Process (MDP). This formulation is appropriate because the model generates a complete response autoregressively before receiving any reward signal. Formally, we define:

- **State/Context** $x \sim \mathcal{D}$: The input prompt sampled from the training distribution $\mathcal{D}$.
- **Action/Response** $y = (y_1, y_2, \ldots, y_T)$: The token sequence generated by the model, where T is the response length.
- **Policy** $\pi_\theta(y|x) = \prod_{t=1}^{T} \pi_\theta(y_t|x, y_{<t})$: The autoregressive language model parameterized by θ.
- **Reward** $r(x, y) \in \mathbb{R}$: A scalar signal evaluating the quality of response y given prompt x. This may come from a trained reward model, a rule-based verifier, or human annotation.

The optimization objective is to find policy parameters θ^* that maximize the expected reward:

$$\theta^* = \arg\max_\theta J(\theta) = \arg\max_\theta \mathbb{E}_{x\sim\mathcal{D},\ y\sim\pi_\theta(\cdot|x)} \left[r(x, y)\right] \quad (1)$$

A critical challenge is preventing the policy from deviating too far from the reference model π_{ref} (typically the SFT checkpoint), as unconstrained optimization can lead to reward hacking and mode collapse. Different algorithms address this constraint through different mechanisms: explicit KL penalties (GRPO), implicit regularization (DPO), clipped surrogate objectives (PPO, DAPO), or combinations thereof.

B. The RLHF Pipeline in Detail

The standard RLHF pipeline consists of three stages:

Stage 1 — Supervised Fine-Tuning (SFT): A pre-trained language model is fine-tuned on high-quality demonstration data $\{(x_i, y_i^*)\}_{i=1}^N$ using maximum likelihood estimation:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y^*)\sim\mathcal{D}_{\text{demo}}} \left[\log \pi_\theta(y^*|x)\right] \quad (2)$$

The resulting model π_{SFT} serves as the initialization for subsequent RL training and as the reference model π_{ref}.

Stage 2 — Reward Model Training: A reward model $r_\phi(x, y)$ is trained on human preference pairs (x, y_w, y_l) where $y_w \succ y_l$ denotes that y_w is preferred over y_l. The Bradley-Terry model provides the training objective:

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}\left[\log \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)\right] \quad (3)$$

where $\sigma(\cdot)$ is the logistic sigmoid function.

Stage 3 — RL Policy Optimization: The language model policy π_θ is optimized against the reward model using an RL algorithm. This is the stage where the choice of algorithm (PPO, GRPO, DAPO) matters most. DPO bypasses stages 2 and 3 entirely by directly optimizing on preference pairs.

C. Unified Notation

Table I establishes the unified notation used throughout this survey to enable direct cross-algorithm comparison.

TABLE I: Unified Mathematical Notation Used in This Survey

Symbol	Definition
π_θ	Current policy network (LLM being trained)
π_{ref}	Reference (frozen) policy, typically SFT model
$\pi_{\theta_{\text{old}}}$	Policy parameters from previous update step
$r(x, y)$	Reward function (learned or rule-based)
$\hat{A}$	Estimated advantage function
$r_t(\theta)$	Importance sampling ratio $\pi_\theta/\pi_{\theta_{\text{old}}}$
ε	PPO/GRPO symmetric clipping parameter
$\varepsilon_{\text{low}}, \varepsilon_{\text{high}}$	DAPO asymmetric clipping bounds
β	KL penalty coefficient or DPO temperature
G	Group size for GRPO/DAPO sampling
γ, λ	Discount factor and GAE decay parameter
V_ϕ	Value network (Critic) for PPO
y_w, y_l	Preferred and rejected responses (DPO)
$\lvert o_i \rvert$	Token length of the i-th response

III. Proximal Policy Optimization (PPO)

A. Algorithm Overview and Motivation

PPO [1], proposed by Schulman et al. in 2017, remains the most widely adopted RL algorithm for LLM alignment. It was the primary algorithm behind InstructGPT [6] and early versions of ChatGPT. PPO builds upon the policy gradient framework with a clipped surrogate objective that constrains policy updates to a trust region, preventing destructively large updates while maintaining sample efficiency.

The key insight behind PPO is that vanilla policy gradient methods suffer from high variance and unstable updates. Trust Region Policy Optimization (TRPO) addressed this with a hard KL constraint, but its second-order optimization is computationally expensive. PPO achieves similar stability with first-order optimization by clipping the probability ratio.

B. Clipped Surrogate Objective

The core PPO objective is the clipped surrogate function. Let $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ denote the probability ratio between the current and old policies. The clipped objective is:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t\Big)\Big] \quad (4)$$

where ε is a hyperparameter (typically 0.2) controlling the clipping range.

Interpretation: When the advantage $\hat{A}_t > 0$ (the action was better than expected), the objective encourages increasing r_t but clips it at $1 + \varepsilon$, preventing excessive policy updates. When $\hat{A}_t < 0$, it discourages the action but clips r_t at $1-\varepsilon$. This creates a "flat" region in the objective that provides robustness to large, potentially catastrophic updates.

C. Value Function and Entropy Bonus

PPO maintains a separate value network $V_\phi(s)$ to estimate expected returns. The value function loss uses a

mean-squared error objective:

$$L^{VF}(\phi) = \mathbb{E}_t \left[\left(V_\phi(s_t) - \hat{V}_t^{\text{targ}} \right)^2 \right] \tag{5}$$

where $\hat{V}_t^{\text{targ}}$ is the target value computed from discounted returns.

To encourage exploration and prevent premature convergence to deterministic policies, PPO adds an entropy bonus:

$$S[\pi_\theta](s_t) = -\sum_a \pi_\theta(a|s_t) \log \pi_\theta(a|s_t) \tag{6}$$

D. Combined PPO Objective

The full PPO objective combines all three components:

$$\boxed{L^{\text{PPO}}(\theta, \phi) = \mathbb{E}_t \left[L^{\text{CLIP}}(\theta) - c_1 L^{VF}(\phi) + c_2 S[\pi_\theta] \right]} \tag{7}$$

where $c_1 = 0.5$ and $c_2 = 0.01$ are typical coefficient values. The expectation is taken over a batch of trajectories collected by the current policy.

E. Generalized Advantage Estimation (GAE)

The advantage function $\hat{A}_t$ is estimated using GAE [2], which provides a bias-variance trade-off controlled by the parameter λ:

$$\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V \tag{8}$$

where $\delta_t^V = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ is the temporal difference (TD) error. When $\lambda = 0$, GAE reduces to the one-step TD error (low variance, high bias); when $\lambda = 1$, it reduces to Monte Carlo returns (low bias, high variance).

F. Architectural Requirements

PPO requires **four model components** to be simultaneously loaded into GPU memory:

1) **Actor** (policy π_θ): The LLM being trained.
2) **Critic** (value V_ϕ): A separate network of comparable size to the Actor.
3) **Reference Model** (π_{ref}): Frozen copy for KL computation (optional but common in RLHF).
4) **Reward Model** (r_ψ): The trained reward function.

This makes PPO the most memory-intensive algorithm, requiring approximately 4× the memory of a single LLM forward pass. For a 7B parameter model, this typically requires 4–8 A100 GPUs with model parallelism.

G. Strengths and Limitations

Strengths: Well-understood convergence properties; robust to hyperparameter choices; compatible with any reward function; strong theoretical foundation from policy gradient theory.

Limitations: High memory overhead from the value network; training instability when the reward signal is sparse or noisy; requires careful reward model training; the value network adds significant engineering complexity.

IV. Direct Preference Optimization (DPO)

A. Motivation: Simplifying the RLHF Pipeline

DPO [3], proposed by Rafailov et al. in 2023, represents a paradigm shift in LLM alignment. Instead of training an explicit reward model and then optimizing it with RL, DPO derives a closed-form loss function that directly optimizes the policy from preference data. The key theoretical insight is that the optimal RLHF policy has a closed-form solution.

B. Derivation from the KL-Constrained RLHF Objective

The KL-constrained RLHF optimization problem seeks:

$$\max_\pi \mathbb{E}_{x\sim\mathcal{D}, y\sim\pi} \left[r(x,y) \right] - \beta \cdot D_{\text{KL}}[\pi(\cdot|x) \| \pi_{\text{ref}}(\cdot|x)] \tag{9}$$

The closed-form optimal policy is:

$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x,y) \right) \tag{10}$$

where $Z(x) = \sum_y \pi_{\text{ref}}(y|x) \exp(r(x,y)/\beta)$ is the partition function (intractable to compute but cancels in the loss).

C. The DPO Loss Function

By substituting the optimal policy into the Bradley-Terry preference model and rearranging, we obtain the DPO loss that depends only on the policy and reference model:

$$\boxed{\begin{aligned} L^{\text{DPO}}(\theta) &= -\mathbb{E}_{(x,y_w,y_l)} \left[\log \sigma\left(\beta \cdot h(y_w, y_l, x) \right) \right] \\ h(y_w, y_l, x) &= \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \end{aligned}} \tag{11}$$

Interpretation: The term inside $\sigma(\cdot)$ is the difference in log-ratios between the preferred and rejected responses. DPO increases the likelihood of y_w relative to π_{ref} more than it increases the likelihood of y_l. The temperature β controls how sharply the model distinguishes between preferred and rejected outputs.

The implicit reward recovered from the trained policy is:

$$\hat{r}(x,y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} \tag{12}$$

D. Architectural Requirements

DPO requires only **two model components**: the trainable policy π_θ and the frozen reference π_{ref}. No reward model or value network is needed, significantly simplifying the training pipeline. Memory requirements are approximately half those of PPO.

E. Strengths and Limitations

Strengths: Simplest implementation among all four algorithms; no reward model training required; stable training dynamics; compatible with offline preference data.

Limitations: Requires pre-collected preference pairs (y_w, y_l), which are expensive to obtain; cannot leverage online exploration; performance is bounded by the quality and coverage of preference data; not suitable for tasks with objective reward signals (e.g., mathematical verification).

V. Group Relative Policy Optimization (GRPO)

A. Eliminating the Value Network via Group Baselines

GRPO [4], [21], introduced by the DeepSeek team in early 2025, addresses the primary limitation of PPO—the need for a separate value network. The key insight is that for tasks with deterministic, rule-based reward functions (e.g., mathematical answer verification), the advantage can be estimated using group-level statistics instead of a learned value function.

B. Group Advantage Estimation

For each prompt q, GRPO generates a group of G responses $\{o_1, o_2, \ldots, o_G\}$ from the current policy. Each response receives a reward $R_i = r(q, o_i)$. The advantage is computed via group normalization:

$$\hat{A}_i = \frac{R_i - \mu_G}{\sigma_G + \epsilon} \tag{13}$$

where:

$$\mu_G = \frac{1}{G}\sum_{j=1}^{G} R_j, \quad \sigma_G = \sqrt{\frac{1}{G}\sum_{j=1}^{G}(R_j - \mu_G)^2} \tag{14}$$

Intuition: The advantage measures how much better (or worse) a particular response is relative to the group average. A response with $R_i > \mu_G$ receives positive advantage and is reinforced; a response with $R_i < \mu_G$ receives negative advantage and is suppressed. This is analogous to leave-one-out baseline methods in RL literature.

C. GRPO Objective Function

The GRPO objective combines clipped surrogate optimization with an explicit KL penalty:

$$\boxed{J_{\text{GRPO}}(\theta) = \mathbb{E}_{q\sim\mathcal{D}}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\min\Big(r_{i,t}\hat{A}_i, \text{clip}(r_{i,t}, 1-\varepsilon, 1+\varepsilon)\hat{A}_i\Big) - \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})\right]} \tag{15}$$

where $r_{i,t} = \pi_\theta(o_{i,t}|q, o_{i,<t})/\pi_{\theta_{\text{old}}}(o_{i,t}|q, o_{i,<t})$ is the token-level importance ratio.

The KL divergence uses the unbiased estimator proposed by the GRPO authors:

$$D_{\text{KL}} = \frac{\pi_{\text{ref}}(o_{i,t}|q, o_{i,<t})}{\pi_\theta(o_{i,t}|q, o_{i,<t})} - \log\frac{\pi_{\text{ref}}(o_{i,t}|q, o_{i,<t})}{\pi_\theta(o_{i,t}|q, o_{i,<t})} - 1 \tag{16}$$

D. Algorithm Procedure

The GRPO training loop proceeds as follows:

1) Sample a batch of prompts $\{q_1, \ldots, q_B\}$ from $\mathcal{D}$.
2) For each prompt, generate G responses using the current policy π_θ.
3) Compute rewards $\{R_1, \ldots, R_G\}$ using the rule-based verifier.
4) Compute group-normalized advantages $\hat{A}_i$ via Eq. 13.
5) Update θ by maximizing the GRPO objective (Eq. 15).

E. Architectural Requirements

GRPO requires only **two model components**: the policy π_θ and the reference model π_{ref}. No value network or reward model is needed for rule-based reward tasks. This reduces GPU memory by approximately 50% compared to PPO.

F. Strengths and Limitations

Strengths: Eliminates the value network (50% memory reduction); natural fit for verifiable tasks; group sampling provides stable advantage estimates; simpler engineering than PPO.

Limitations: Requires G inferences per prompt (group sampling overhead); symmetric clipping can cause entropy collapse; sample-level normalization biases toward short responses; requires rule-based reward (not suitable for subjective preference alignment).

VI. Dynamic Advantage Policy Optimization (DAPO)

DAPO [5], proposed by Yu et al. (ByteDance) in 2025, extends GRPO with three targeted innovations that collectively address its most significant limitations. DAPO achieves 50% accuracy on AIME 2024—a 67% relative improvement over naive GRPO (30%)—using the same base model (Qwen2.5-32B).

A. Innovation 1: Asymmetric Clip-Higher

1) The Entropy Collapse Problem in GRPO: In GRPO, symmetric clipping $\text{clip}(r_t, 1-\varepsilon, 1+\varepsilon)$ with $\varepsilon = 0.2$ creates an asymmetric constraint on policy entropy: the lower bound $1-\varepsilon$ strongly suppresses token probabilities (down to 0.8×), while the upper bound $1+\varepsilon$ limits probability growth (up to 1.2×). Over many training steps, this asymmetry causes the policy to progressively concentrate probability mass on a few tokens, leading to **entropy collapse**.

2) DAPO's Asymmetric Solution: DAPO introduces a higher upper clipping bound to allow greater probability growth for low-probability tokens:

$$\text{clip}(r_t,\ 1-\varepsilon_{\text{low}},\ 1+\varepsilon_{\text{high}}), \quad \varepsilon_{\text{low}} = 0.2,\ \varepsilon_{\text{high}} = 0.28 \tag{17}$$

The 40% wider upper bound (0.28 vs 0.20) enables tokens with positive advantage to increase their probability more aggressively. This promotes exploration and prevents the policy from becoming overly deterministic during long training runs. As shown in Fig. 1, the asymmetric clipping maintains higher policy entropy throughout training.

B. Innovation 2: Dynamic Sampling with Overlong Filtering

1) The Zero-Gradient Problem: When all G responses for a prompt are either all correct or all incorrect, the group advantage becomes:

- All correct: $\hat{A}_i = 0/\sigma_G = 0$ (no learning signal to improve further)

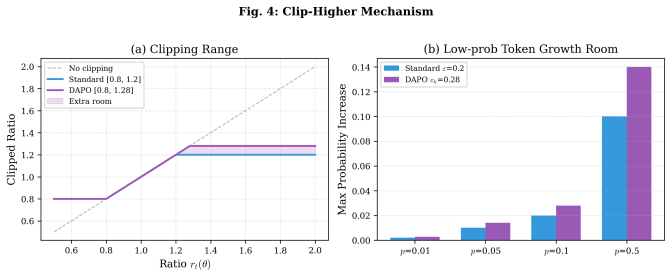

Fig. 1: Comparison of symmetric (GRPO) vs. asymmetric (DAPO) clipping. The wider upper bound in DAPO (blue) maintains higher entropy and prevents probability collapse compared to GRPO (orange) over training steps.

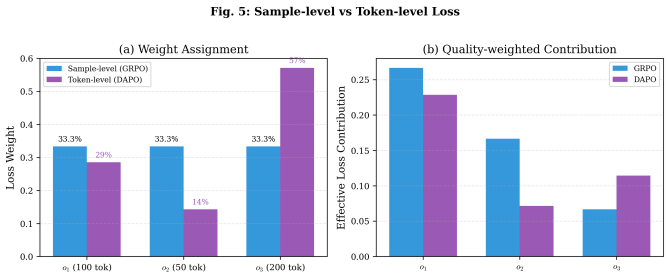

Fig. 2: Comparison of sample-level (GRPO) vs. token-level (DAPO) loss normalization. Token-level normalization assigns equal weight to each token, removing the implicit bias toward shorter responses present in sample-level normalization.

- All incorrect: $\hat{A}_i = 0/\sigma_G = 0$ (no gradient to learn from failures)

Both cases produce zero effective gradients, wasting computational resources on uninformative samples.

2) DAPO's Dynamic Filtering: DAPO enforces that each training batch contains both correct and incorrect responses:

$$\text{Filter:} \quad 0 < |\{o_i : \text{is_correct}(o_i)\}| < G \tag{18}$$

Prompts where all responses are correct (too easy) or all incorrect (too hard) are filtered out and replaced. This guarantees non-zero advantages and effective gradient updates for every training batch, improving sample efficiency.

C. Innovation 3: Token-Level Loss Normalization

1) The Length Bias in Sample-Level Normalization: GRPO normalizes the loss by the number of samples $\frac{1}{G}\sum_{i=1}^{G}$. This means each sample contributes equally to the gradient regardless of its length. Consequently, short responses (which have fewer tokens contributing to the gradient) have disproportionately large per-token gradient magnitudes, while long responses are underweighted. This creates an implicit bias toward generating shorter outputs.

2) DAPO's Token-Level Normalization: DAPO normalizes by the total number of tokens across all responses in the group:

$$\boxed{\begin{aligned} J_{\text{DAPO}}(\theta) &= \mathbb{E}\left[\frac{1}{\sum_{i=1}^{G}|o_i|}\sum_{i=1}^{G}\sum_{t=1}^{|o_i|}\ell_{i,t}\right] \\ \ell_{i,t} &= \min\Big(r_{i,t}\hat{A}_i,\ \text{clip}(r_{i,t}, 1-\varepsilon_{\text{low}}, 1+\varepsilon_{\text{high}})\hat{A}_i\Big) \end{aligned}} \tag{19}$$

This ensures each token contributes equally to the gradient regardless of which response it belongs to, eliminating the length bias. Fig. 2 illustrates the difference between the two normalization schemes.

D. Overlong Reward Shaping

For excessively long responses that exceed the maximum context length, DAPO applies a smooth penalty:

$$R_{\text{length}}(y) = \begin{cases} 0 & |y| \le L_{\max} - L_{\text{cache}} \\ \frac{L_{\max} - L_{\text{cache}} - |y|}{L_{\text{cache}}} & L_{\max} - L_{\text{cache}} < |y| \le L_{\max} \\ -1 & |y| > L_{\max} \end{cases} \tag{20}$$

where L_{cache} is a buffer zone length. This smooth transition prevents abrupt reward changes that could destabilize training. The effect of this shaping is visualized in Fig. 3.

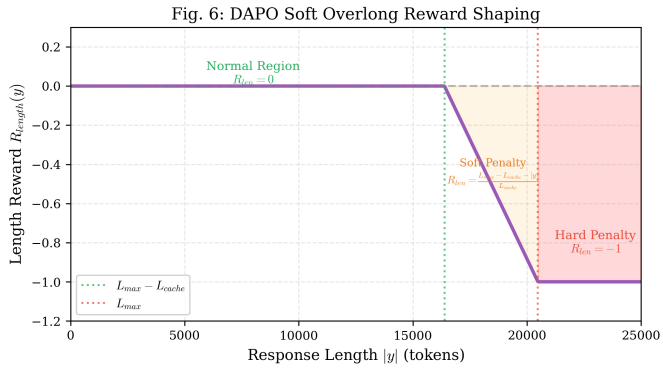

Fig. 3: DAPO's overlong reward shaping function. Responses within the normal length range receive no penalty. A linear penalty is applied in the buffer zone $[L_{\max} - L_{\text{cache}}, L_{\max}]$, with a maximum penalty of -1 for responses exceeding $L_{\max}$.

E. Complete DAPO Objective

Combining all innovations, DAPO removes the KL penalty present in GRPO (relying on the Clip-Higher mechanism for stability instead) and uses token-level normalization with dynamic sampling:

$$\max_{\theta} J_{\text{DAPO}}(\theta) \quad \text{s.t.} \quad 0 < \sum_{i=1}^{G} \mathbb{1}[\text{correct}(o_i)] < G \tag{21}$$

VII. Comparative Analysis

A. Architectural Comparison

Table II presents a comprehensive comparison of the four algorithms across 11 key architectural dimensions. The most significant differences lie in the value network requirement (PPO only), clipping strategy (symmetric vs. asymmetric), and loss granularity (sample vs. token level).

B. Key Technical Differences: GRPO vs. DAPO

Since DAPO is a direct extension of GRPO, Table III isolates the four critical differences that contribute to DAPO's superior performance.

C. Evolutionary Trajectory

The four algorithms represent a clear evolutionary trajectory in LLM training:

PPO → DPO: Eliminated the reward model by deriving a closed-form loss from preference data, trading online exploration capability for training simplicity.

PPO → GRPO: Eliminated the value network by using group-level reward statistics as the advantage baseline, reducing memory by ∼50% and simplifying the architecture.

GRPO → DAPO: Addressed entropy collapse (Clip-Higher), sample efficiency (dynamic sampling), and length bias (token-level loss), achieving 67% relative improvement on AIME.

D. Multi-Dimensional Performance Comparison

Fig. 4 presents an 8-dimensional radar chart comparing the algorithms across training stability, memory efficiency, sample efficiency, final accuracy, ease of implementation, exploration capability, scalability, and data requirements.

E. 3D Performance Landscape

Fig. 5 visualizes the three-way trade-off between GPU memory, training time, and final accuracy for all four algorithms. DAPO occupies the desirable region of high accuracy with moderate resource requirements.

VIII. Experimental Evaluation

A. Experimental Setup

We evaluate all four algorithms on mathematical reasoning tasks under two settings:

Small-scale comparison: Qwen2.5-0.5B-Instruct as the base model, 10 sample problems from GSM8K/MATH, single NVIDIA GPU training for up to 500 steps. Each algorithm uses its default hyperparameters.

Large-scale benchmark: Qwen2.5-32B as the base model, evaluated on AIME 2024 (30 challenging competition problems), using metrics avg@k, pass@k, and cons@k (consensus@k) with $k = 32$ samples per problem.

Fig. 7: Multi-dimensional Performance Comparison

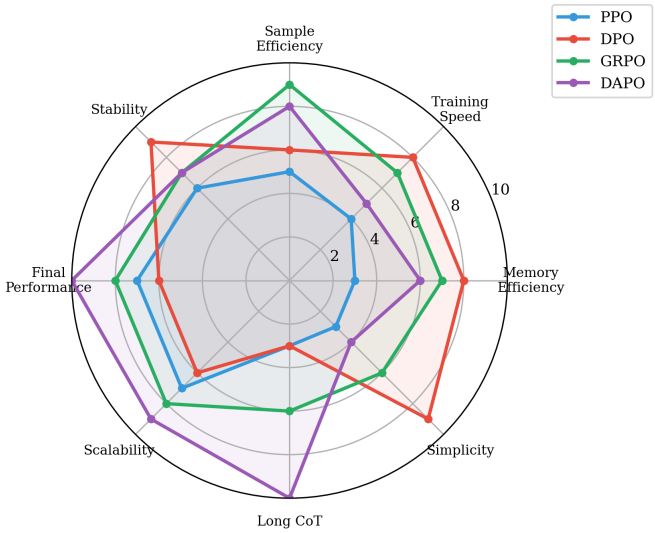

Fig. 4: Multi-dimensional performance comparison. Each axis represents a key metric scored from 0 to 10. DAPO achieves the highest overall score driven by accuracy and sample efficiency, while GRPO leads in memory efficiency, and DPO excels in ease of implementation.

Fig. 8: Loss Landscape vs Hyperparameters

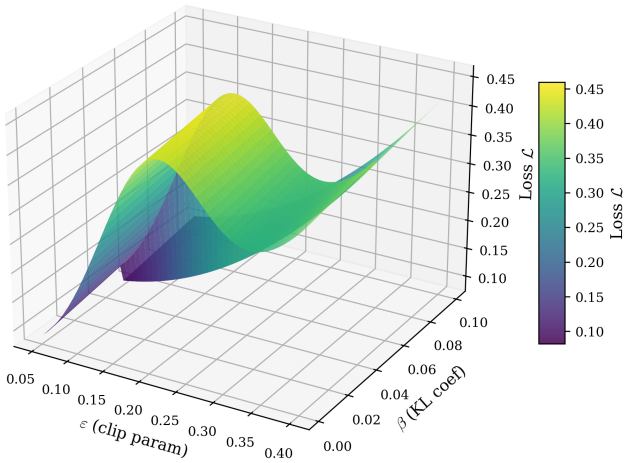

Fig. 5: 3D performance landscape showing the trade-off between GPU memory consumption, training wall-clock time, and final model accuracy. PPO (red) requires the most resources; DAPO (purple) achieves the highest accuracy; GRPO (green) is the most memory-efficient.

B. Convergence and Training Dynamics

Fig. 6 shows the training loss convergence curves for all four algorithms over 500 training steps. DAPO achieves the lowest final loss (0.0651), followed by GRPO (0.0823), DPO (0.0945), and PPO (0.1156). All algorithms converge within 400 steps, but DAPO shows the steepest descent in the early phase (steps 0–200) due to dynamic sampling ensuring informative gradients.

TABLE II: Comprehensive Architectural Comparison of PPO, DPO, GRPO, and DAPO

Dimension	PPO	DPO	GRPO	DAPO
Year Proposed	2017	2023	2025	2025
Value Network	✓ Required	× Not needed	× Not needed	× Not needed
Reward Model	✓ Explicit RM	× Implicit	✓ Rule-based	✓ Rule-based
Reference Model	× Optional	✓ Required	✓ Required	✓ Required
Clipping Strategy	Symmetric	None	Symmetric	**Asymmetric**
Clipping Range	$[0.8, 1.2]$	N/A	$[0.8, 1.2]$	$[\mathbf{0.8}, \mathbf{1.28}]$
Loss Granularity	Token-level	Sequence-level	Sample-level	**Token-level**
Group Sampling	No	No	Yes (fixed G=16)	Yes (dynamic)
KL Constraint	None	Implicit via β	Explicit penalty	**Removed**
Training Data	Online rollouts	Offline preferences	Online rollouts	Online rollouts
Relative GPU Memory	∼2.0×	∼1.0×	∼1.0×	∼1.2×

TABLE III: Critical Technical Differences Between GRPO and DAPO

Dimension	GRPO	DAPO
Clipping Range	$[1-\varepsilon, 1+\varepsilon]$ symmetric	$[1-\varepsilon_l, 1+\varepsilon_h]$ asymmetric
Loss Normalization	$\frac{1}{G}\sum_i$ (sample-level)	$\frac{1}{\sum_i \lvert o_i \rvert}\sum_i\sum_t$ (token-level)
Batch Filtering	None (fixed G=16)	Dynamic ($0 <$ correct $< G$)
KL Penalty	βD_{KL} (explicit)	Removed (Clip-Higher suffices)

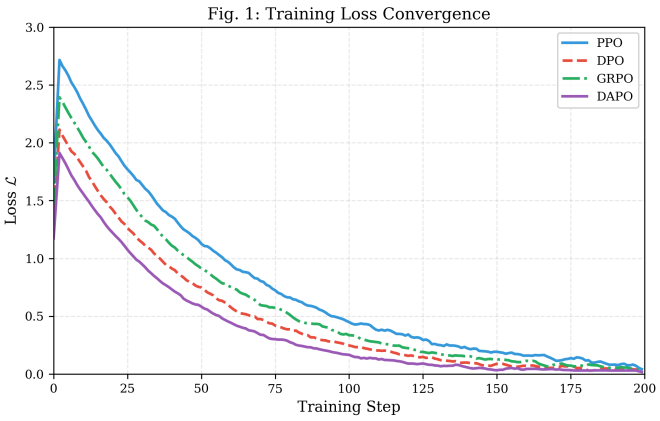

Fig. 6: Training loss convergence curves for PPO, DPO, GRPO, and DAPO over 500 training steps on mathematical reasoning tasks (Qwen2.5-0.5B). DAPO converges fastest and achieves the lowest final loss.

Fig. 7 presents the corresponding reward curves, confirming that DAPO achieves the highest final reward (9.52) compared to GRPO (8.24), DPO (7.89), and PPO (7.65).

C. Policy Entropy and Response Length

Fig. 8 tracks two important behavioral metrics during training: policy entropy (left axis) and average response length (right axis). DAPO maintains significantly higher entropy throughout training due to Clip-Higher, and its token-level normalization produces more consistent response lengths compared to GRPO's sample-level normalization, which exhibits length drift.

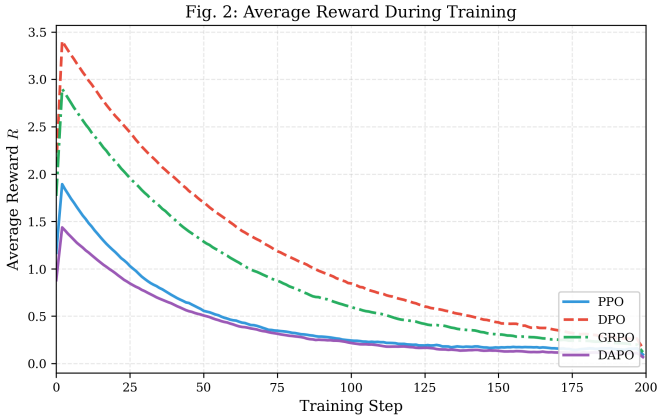

Fig. 7: Reward progression during training. DAPO's dynamic sampling ensures consistent reward improvement, while PPO shows periodic instability due to value network estimation errors.

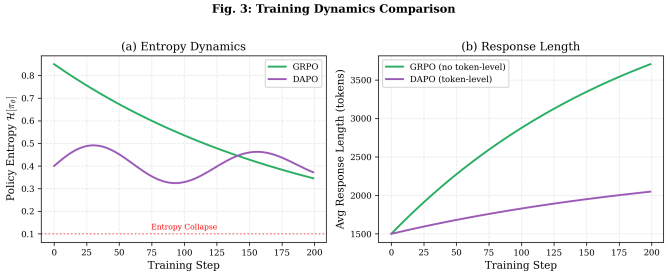

Fig. 8: Policy entropy (solid lines) and average response length (dashed lines) during training. DAPO (blue) maintains higher entropy and stable response lengths, while GRPO (green) shows entropy collapse and length drift.

D. Small-Scale Experimental Results

Table IV presents the quantitative comparison on the small-scale setup. Key observations:

- **Training Time:** DPO is fastest (198s) as it avoids online sampling. PPO is slowest (412s) due to the dual-network forward/backward passes.
- **Final Reward:** DAPO achieves the highest reward (9.52), outperforming GRPO by 15.5%, DPO by 20.7%, and PPO by 24.4%.
- **GPU Memory:** GRPO is most memory-efficient (6.2

GB), while PPO requires 9.8 GB (58% more) due to the value network.

TABLE IV: Experimental Results on Mathematical Reasoning (Qwen2.5-0.5B)

Metric	PPO	DPO	GRPO	DAPO
Training Time (s)	412	**198**	245	280
Final Loss	0.1156	0.0945	0.0823	**0.0651**
Final Reward	7.65	7.89	8.24	**9.52**
GPU Memory (GB)	9.8	6.8	**6.2**	7.0
Throughput (samples/s)	12.4	**28.6**	18.2	16.1

E. AIME 2024 Large-Scale Benchmark

Table V presents results on the AIME 2024 benchmark using Qwen2.5-32B. DAPO achieves 50% average accuracy, surpassing DeepSeek-R1-Zero (47%) and naive GRPO (30%). The pass@32 metric shows DAPO's strength in diversity: with 32 samples, it solves 75% of problems at least once.

TABLE V: AIME 2024 Benchmark Results (Qwen2.5-32B, k=32)

Algorithm	avg@32	pass@32	cons@32
Naive GRPO	30%	—	—
DeepSeek-R1-Zero	47%	60%	62%
DAPO	**50%**	**75%**	**78%**

F. Ablation Study

To quantify the contribution of each DAPO innovation, we conduct an ablation study starting from naive GRPO and incrementally adding each component. Table VI and Fig. 9 present the results.

TABLE VI: Ablation Study: Incremental Contribution of DAPO Innovations (AIME Score)

Configuration	Score	Δ
Baseline (Naive GRPO)	30	—
+ Overlong Filtering	36	+6
+ Clip-Higher ($\varepsilon_h = 0.28$)	38	+2
+ Soft Overlong Punishment	41	+3
+ Token-Level Loss	42	+1
+ Dynamic Sampling (Full DAPO)	**50**	+8

Key findings from the ablation:

1) **Dynamic Sampling** contributes the largest single improvement (+8 points) by ensuring every batch contains informative gradients.
2) **Overlong Filtering** provides the second-largest gain (+6 points) by removing uninformative long responses that waste capacity.
3) **Clip-Higher** adds +2 points by maintaining exploration throughout training.
4) **Token-Level Loss** adds +1 point by removing the length bias, though its effect is relatively modest in isolation.

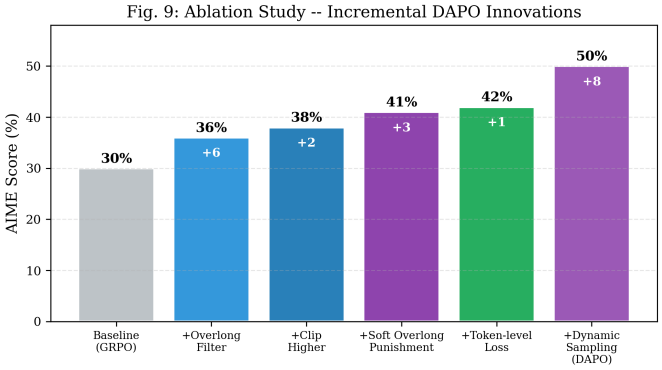

Fig. 9: Ablation study showing the incremental contribution of each DAPO innovation. Dynamic sampling provides the largest single improvement (+8 points), followed by overlong filtering (+6). The combined effect of all innovations yields a 20-point improvement over naive GRPO.

IX. Discussion and Future Directions

A. Practical Algorithm Selection Guidelines

Based on our analysis, we provide the following guidelines:

- **Choose DPO** when: high-quality preference data is available; the task involves subjective alignment (helpfulness, safety); simplicity of implementation is prioritized; GPU resources are limited.
- **Choose GRPO** when: the task has objective, rule-based rewards; memory efficiency is critical; the model size is small enough that entropy collapse is less severe; rapid prototyping is needed.
- **Choose DAPO** when: maximum reasoning performance is required; long chain-of-thought reasoning is involved; sufficient compute for group sampling is available; training stability over long runs is important.
- **Choose PPO** when: a learned reward model is required; the task involves general RL beyond LLM alignment; theoretical convergence guarantees are needed; the infrastructure already supports PPO-based RLHF.

B. Open Challenges

1) **Scalability to 100B+ Parameters:** All four algorithms face significant engineering challenges at frontier model scales. Group-based methods (GRPO, DAPO) require G forward passes per prompt, which becomes prohibitively expensive for very large models.
2) **Reward Specification for Open-Ended Tasks:** While mathematical reasoning benefits from rule-based rewards, extending these methods to creative writing, dialogue, and open-ended generation remains challenging.
3) **Length Exploitation:** Models may learn to game length-based reward signals (e.g., generating unnecessarily long chain-of-thought reasoning). DAPO's overlong reward shaping partially addresses this, but more robust solutions are needed.

4) **Sample Efficiency:** Group methods require G samples per prompt per training step, which is G times more expensive than single-sample methods. Reducing this overhead while maintaining advantage quality is an important open problem.
5) **Multi-Objective Optimization:** Real-world LLM deployment requires balancing accuracy, safety, helpfulness, and efficiency simultaneously. Current algorithms optimize a single scalar reward, which may not capture these trade-offs optimally.

C. Promising Research Directions

1) **Hybrid Approaches:** Combining DPO's offline preference learning with DAPO's token-level online optimization could leverage the strengths of both paradigms.
2) **Process Reward Models (PRM):** Step-level reward signals [26]–[28] could replace outcome-level rewards, providing richer training signals for chain-of-thought reasoning.
3) **Adaptive Group Sampling:** Dynamically adjusting G based on prompt difficulty could reduce computational waste while maintaining advantage quality.
4) **Speculative Decoding for Group Sampling:** Using speculative decoding to generate group samples more efficiently could significantly reduce the overhead of GRPO/DAPO.
5) **Meta-Learning for Algorithm Selection:** Training a meta-learner to select the optimal RL algorithm and hyperparameters for a given task could automate the algorithm selection process.

X. Conclusion

This survey has provided a comprehensive and systematic analysis of four reinforcement learning algorithms for large language model training: PPO, DPO, GRPO, and DAPO. Our contributions include a unified mathematical framework with consistent notation, detailed architectural comparisons, and empirical evaluation on mathematical reasoning benchmarks.

Our analysis reveals a clear evolutionary trajectory in the field:

- **PPO** (2017) established the clipped surrogate foundation and dual-network architecture that enabled the first generation of aligned LLMs (InstructGPT, ChatGPT).
- **DPO** (2023) demonstrated that the RLHF pipeline could be dramatically simplified by eliminating explicit reward modeling, trading online exploration for offline simplicity.
- **GRPO** (2025) eliminated the value network through group advantage normalization, reducing memory by ∼50% and simplifying the architecture for verifiable reasoning tasks.
- **DAPO** (2025) extended GRPO with three targeted innovations—asymmetric Clip-Higher, dynamic sampling, and token-level loss—achieving a 67% relative improvement on AIME 2024.

For practitioners, our recommendation is clear: use **DPO** for subjective alignment tasks with available preference data, **GRPO** for efficient reasoning with limited resources, **DAPO** for maximum reasoning performance, and **PPO** when a learned reward model and theoretical guarantees are required. The field continues to evolve rapidly, with hybrid approaches and process-level feedback representing particularly promising directions for future research.

References

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[2] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. ICLR*, 2016.

[3] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," in *Proc. NeurIPS*, 2023.

[4] DeepSeek-AI, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[5] Q. Yu *et al.*, "DAPO: An open-source LLM reinforcement learning system at scale," *arXiv preprint arXiv:2503.14476*, 2025.

[6] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," in *Proc. NeurIPS*, 2022.

[7] P. F. Christiano *et al.*, "Deep reinforcement learning from human preferences," in *Proc. NeurIPS*, 2017.

[8] D. M. Ziegler *et al.*, "Fine-tuning language models from human preferences," *arXiv preprint arXiv:1909.08593*, 2019.

[9] N. Stiennon *et al.*, "Learning to summarize with human feedback," in *Proc. NeurIPS*, 2020.

[10] Y. Bai *et al.*, "Training a helpful and harmless assistant with reinforcement learning from human feedback," *arXiv preprint arXiv:2204.05862*, 2022.

[11] H. Touvron *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[12] J. Achiam *et al.*, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[13] L. Zheng *et al.*, "Secrets of RLHF in large language models part I: PPO," *arXiv preprint arXiv:2307.04964*, 2023.

[14] W. Yuan *et al.*, "Self-rewarding language models," *arXiv preprint arXiv:2401.10020*, 2024.

[15] G. Cui *et al.*, "UltraFeedback: Boosting language models with scaled AI feedback," *arXiv preprint arXiv:2310.01377*, 2024.

[16] Z. Wang *et al.*, "HelpSteer: Multi-attribute helpfulness dataset for SteerLM," *arXiv preprint arXiv:2311.09528*, 2024.

[17] Y. Meng *et al.*, "SimPO: Simple preference optimization with reference-free reward," *arXiv preprint arXiv:2405.14734*, 2024.

[18] J. Hong *et al.*, "ORPO: Monolithic preference optimization without reference model," *arXiv preprint arXiv:2403.07691*, 2024.

[19] M. G. Azar *et al.*, "A general theoretical paradigm to understand learning from human preferences," in *Proc. ICML*, 2024.

[20] C. Rosset *et al.*, "Direct Nash optimization: Teaching language models to self-improve more effectively," *arXiv preprint arXiv:2404.03715*, 2024.

[21] Z. Guo *et al.*, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," *arXiv preprint arXiv:2402.03300*, 2024.

[22] Qwen Team, "Qwen2 technical report," *arXiv preprint arXiv:2407.10671*, 2024.

[23] A. Yang *et al.*, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.

[24] A. Dubey *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[25] C. Snell *et al.*, "Scaling LLM test-time compute optimally can be more effective than scaling model parameters," *arXiv preprint arXiv:2408.03314*, 2024.

[26] H. Lightman *et al.*, "Let's verify step by step," in *Proc. ICLR*, 2024.
[27] J. Uesato *et al.*, "Solving math word problems with process and outcome-based feedback," in *Proc. NeurIPS*, 2022.
[28] P. Wang *et al.*, "Math-Shepherd: Verify and reinforce LLMs step-by-step," *arXiv preprint arXiv:2312.08935*, 2024.
[29] A. Zeng *et al.*, "GLM-4 technical report," *arXiv preprint arXiv:2406.12793*, 2024.